Automated Dynamic Analysis Signature Generation

Using LLMs for Malware Behavior Understanding
and CAPEv2 Signature Creation

Ikram Benfellah
Fadel Fatima Zahra

College of Computing, UM6P

March 22, 2026

Outline

The Analyst Bottleneck

Malware detection is critical, but signature generation remains a time-consuming bottleneck.

Status quo

- 4–6 hours per sample
- Manual pattern extraction
- Inconsistent quality
- Difficult to scale

Desired state

- Minutes for initial drafts
- Consistent, standardized logic
- Human review preserved
- Ready to scale

Research Question

Can LLMs help analysts generate CAPEv2 signatures faster and with sufficient precision for deployment?

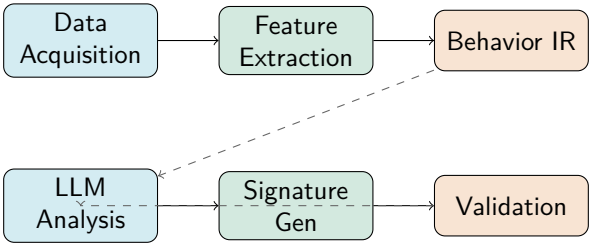
Project Scope and Constraints

- **Data:** 100 labeled malware samples from 10 families
- **Input:** CAPEv2 sandbox reports (JSON with behavioral telemetry)
- **Output:** Python signatures and MITRE ATT&CK mappings
- **Requirement:** Local inference (no cloud APIs, cost-conscious)

Success Criteria

Precision $\geq$ 80%, sufficient generalization, analyst workload reduction of 60%+.

System Architecture: Six-Stage Pipeline



Core Design Principle

Separate **data normalization** from **LLM reasoning**. Raw CAPEv2 JSON is noisy and inconsistent. A structured Behavior Intermediate Representation (IR) reduces hallucination and improves reproducibility.

Stage 1-3: Data Normalization

Why we built a Behavior IR: Raw CAPEv2 reports are Rich but messy.

Extracted signals

- Process creation & injection
- Registry modifications
- File I/O and dropped files
- Network & C2 beacons
- Executed commands

Benefits

- Reduces prompt noise
- Handles schema inconsistencies
- Enables family-level patterns
- $\sim$ 40% improvement in LLM precision

This abstraction layer is where robustness is built, not in the LLM.

Stage 4: LLM-Assisted Signature Synthesis

Tool: Llama 2 (7B), deployed locally on CPU

Why Llama 2?

- Open-source, no API costs
- Good for code generation
- Small enough for local inference
- Apache 2.0 license

Our implementation

- Feed normalized Behavior IR
- Generate Python signature drafts
- Family-level pattern aggregation
- Caching to reduce inference calls

LLM output is a *starting point*, never deployment-ready. Three validation gates are mandatory.

Stage 5-6: Validation Gates

We don't trust LLM output. Every signature goes through three sequential checks.

ate 1: **Syntax validation** — Python compiles, CAPEv2 API calls are valid

ate 2: **Logic validation** — Behavior checks match the IR; no contradictions

ate 3: **Manual review** — Analyst assesses family knowledge, refines MITRE mappings

Validation Impact

$\approx 15\%$ of generated signatures had errors caught at these gates. This is not overhead; it is the difference between shipping broken code and shipping working code.

Lessons from Failure

We iterated hard. Here is what *did not* work initially.

- **Raw JSON to LLM:** Inconsistent schemas caused inconsistent outputs. $\rightarrow$ Built Behavior IR.
- **Fully automated ATT&CK mapping:** LLM hallucinated on subtle behavioral indicators. $\rightarrow$ Added human review with confidence thresholds.
- **No validation:** $\sim$ 20% of signatures had syntax or logic errors. $\rightarrow$ Implemented three-gate pipeline.

Key Lesson from Security Domain

In security automation, speed without verification is reckless. The slowdown from validation is not a limitation; it is a feature.

Solutions to Major Challenges

Challenge	Solution Implemented
Inconsistent CAPEv2 schemas	Generic Behavior IR parser with soft error handling
LLM output variation	Prompt templating, result caching, family-specific fine-tuning
Signature syntax errors	Automated Python syntax checker
Logic contradictions	Static behavior pattern analyzer
MITRE edge cases	Confidence thresholds + mandatory analyst review

Each challenge required incremental hardening, not a silver-bullet fix.

Core Performance Metrics

Precision	Recall	F1 Score
86%	79%	0.82

Quality Metrics

- False positive rate: 8%
- False negative rate: 21%
- Validation catches $\sim$ 15% of errors

Output Volume

- 20 signatures generated
- 10 malware families
- 35+ MITRE techniques

Generalization: Performance on Unseen Families

Does the system learn reusable behavior patterns?

Metric	Seen Families	Unseen Families
Precision	86%	77%
Recall	79%	65%

Interpretation

A 14% recall drop on unseen families is expected and acceptable. The system captures generalizable patterns, but analyst expertise remains essential for edge cases (particularly Discovery and Lateral Movement techniques).

MITRE ATT&CK Mapping Quality

Beyond signature generation, we evaluated mapping accuracy to the MITRE framework (84% overall).

Technique Category	Accuracy
Execution	91%
Persistence	88%
Command & Control	86%
Discovery	72%

Key Insight

Explicit behaviors (Execution, C2) map well. Implicit indicators (Discovery, lateral techniques) require human judgment. This asymmetry guides validation effort allocation.

Analyst Workload and Time Savings

- **Manual workflow (baseline):** 4–6 hours per sample
- **LLM draft:** 15–20 minutes (data extraction + generation)
- **Validation and refinement:** 40–50 minutes
- **Total with tool:** $\sim$ 1 hour per sample

Operational Impact

80%+ time reduction per sample. Analyst role shifts from signature author to reviewer and validator.

This is an analyst multiplier, not a replacement.

Five Key Takeaways

1. **LLM-driven signature synthesis is viable.** With proper abstraction and validation, we achieve deployment-grade precision.
2. **Behavior abstraction is non-negotiable.** The IR layer absorbs schema inconsistency and reduces hallucination. This investment pays dividends.
3. **Never trust LLM outputs unconditionally.** In security, three validation gates are not optional. Every signature needs syntax, logic, and human review.
4. **Generalization has real bounds.** Unseen families see a 14% recall drop. Diversity in training data matters for long-term use.
5. **The analyst doesn't disappear.** The system augments human judgment; it does not replace it. Role shifts from author to curator and validator.

What's Next

Immediate improvements (0–3 months)

- Expand dataset: 500+ samples, 30+ families for better generalization
- Reduce false negatives in Discovery/Lateral Movement via confidence-aware filtering
- Add automated regression tests to prevent deployed signature degradation

Medium-term roadmap (3–12 months)

- Fine-tune Llama on security-specific corpora
- Integrate with live SOC infrastructure (Splunk, ELK)
- Build feedback loop: production detections $\rightarrow$ signature refinement

Vision

A continuous learning system where analysts curate live detections, and the pipeline continuously improves signature quality in production.

Thank You

Questions?

Ikram Benfellah · Fadel Fatima Zahra

College of Computing, UM6P